

Hands-on Activity 2.1: The Tower of Hanoi Problem

Intended Learning Outcomes

- Analyze and solve computational problems using problem solving methodologies and decomposition

Objectives

- Introduce students to the Tower of Hanoi Problem;
- Apply the fundamentals of logical and algorithmic thinking;
- Solve the Tower of Hanoi Problem using Python.

Code:

```
Hands_On_Activity_1_1_Optimization_and_Knapsack_Problem_1.ipynb  Hands_On_Activity_2.1_The_Tower_of_Hanoi_Problem.py M X
Hands_On_Activity_2.1_The_Tower_of_Hanoi_Problem.py > TowerOfHanoi
Run Cell | Run Below | Debug Cell
1  # %%
2  move = 0
3
4  def TowerOfHanoi(n, source, auxiliary, destination):
5      global move
6      if n == 1:
7          move += 1
8          print(f"Move disk 1 from {source} to {destination} (Move {move})")
9          return
10
11      TowerOfHanoi(n-1, source, destination, auxiliary)
12
13      move += 1
14      print(f"Move disk {n} from {source} to {destination} (Move {move})")
15
16      TowerOfHanoi(n-1, auxiliary, source, destination)
17
18  Run Cell | Run Above | Debug Cell
19  # %%
20  def run_TowerOfHanoi(n):
21      global move
22      move = 0
23      TowerOfHanoi(n, "A", "B", "C")
24      print(f"\nTotal moves used: {move}")
25
26  # Driver for 4 disks
27  run_TowerOfHanoi(4)
```

Output:

```
em.py
Move disk 1 from A to B (Move 1)
Move disk 2 from A to C (Move 2)
Move disk 1 from B to C (Move 3)
Move disk 3 from A to B (Move 4)
Move disk 1 from C to A (Move 5)
Move disk 2 from C to B (Move 6)
Move disk 1 from A to B (Move 7)
Move disk 4 from A to C (Move 8)
Move disk 1 from B to C (Move 9)
Move disk 2 from B to A (Move 10)
Move disk 1 from C to A (Move 11)
Move disk 3 from B to C (Move 12)
Move disk 1 from A to B (Move 13)
Move disk 2 from A to C (Move 14)
Move disk 1 from B to C (Move 15)

Total moves used: 15
PS C:\Users\tipqc\CPE-311---Computational-Thinking-with-Python>
```

Explanation:

The Tower of Hanoi program/algorithm uses **recursion**, which basically means the function keeps calling itself to solve smaller parts of the problem until it reaches the simplest case. This follows the **divide and conquer technique**, wherein a big task is broken into smaller tasks that are easier to handle. A **global counter** keeps track of the number of moves so we can see both the step-by-step disk movements and the total moves used. Overall, this program demonstrates how breaking a complex puzzle into smaller steps makes it easier to solve.